LIBRARY MANAGEMENT SYSTEM

CS23531 – WEB PROGRAMMING
MINI PROJECT REPORT

Submitted by

KAMAL SUNDAR S – 2116241001103

Of

BACHELOR OF TECHNOLOGY

In

INFORMATION TECHNOLOGY

RAJALAKSHMI ENGINEERING COLLEGE, THANDALAM

(An Autonomous Institution)



OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project report titled "**LIBRARY MANAGEMENT SYSTEM**" is the bonafide work of "**KAMAL SUNDAR S (2116241001103)"** who carried out the work under my supervision.

SIGNATURE
Dr. P. Neelaveni
HEAD OF THE DEPARTMENT
Information Technology
Rajalakshmi Engineering College,
Rajalakshmi Nagar, Thandalam,
Chennai - 602105

SIGNATURE
Mrs. Janani M
COURSE INCHARGE
Information Technology
Rajalakshmi Engineering College,
Rajalakshmi Nagar, Thandalam,
Chennai - 602105

This project is submitted for CS23531 – Web Programming held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ABSTRACT

The Library Management System is a Java-based application designed to support the day-to-day activities of a library. It provides facilities for maintaining book records, managing borrower information, issuing and returning books, renewing loans, placing hold requests and recording fine-payment details. The system defines four main actors: Borrower, Checkout Clerk, Librarian and Administrator. Each actor is provided with operations according to their responsibilities.

The application follows object-oriented analysis and design principles. It separates domain entities and operational responsibilities while keeping the user interface minimal. It uses Java SE, JDBC and Java DB (Apache Derby) for persistent storage. The database contains tables for persons, books, staff, borrowers, loans, borrowed books and hold requests. The system aims to improve the accuracy of library records, reduce manual effort and simplify book-circulation activities.

This report is based on the public Java Library Management System repository by Haris Muneer and contributors. The original repository describes a console-based interface and a NetBeans/JDK 8 setup.

TABLE OF CONTENTS

S.No	Section Title
1	1.1 Introduction
2	1.2 Purpose
3	1.3 Scope of the Project
4	1.4 Software Requirements Specification
5	2.1 Use Case Diagram
6	2.2 Entity Relationship Diagram
7	2.3 Data Flow Diagram
8	3. Module Description
9	4.1 User Interface and Workflow
10	4.2 Data Design
11	4.3 Selected Source Code
12	4.4 Testing Scenarios
13	5. Conclusion
14	6. References

1. PROJECT OVERVIEW

1.1 INTRODUCTION

Libraries maintain large collections of books and provide services to many borrowers. In a manual system, recording book details, tracking issued books, maintaining borrower information and checking returns can require considerable time. Manual records may also become difficult to search and update as the number of books and borrowers increases.

The Library Management System computerizes these activities through a Java application connected to a relational database. Borrowers can search for books using title, author or subject and view personal information and currently borrowed books. Checkout clerks can issue and receive books, renew loans, add borrowers, update borrower details and record fine payments. Librarians manage the book collection, while administrators add staff members and view library records.

The project follows object-oriented analysis and design. The interface is console-based, while entities and operational classes are separated to reduce coupling and improve maintainability. The repository also provides a class diagram and database schema.

1.2 PURPOSE

- Maintain book details such as title, author, subject and issue status in a structured database.
- Provide book search by title, author or subject.
- Record book issue and return transactions accurately.
- Allow borrowers to renew books and place hold requests for books currently on loan.
- Maintain borrower details and the list of books currently borrowed.
- Support library staff in managing books, borrowers, loans and fine-payment records.
- Allow administrators to add clerks and librarians and view library information.

1.3 SCOPE OF THE PROJECT

- Book search: search the collection by title, author and subject.
- Borrower management: add borrowers and update address or telephone information.
- Book issue: check out an available book and record the transaction.
- Book return: check in a returned book and record return details.
- Loan renewal: renew an existing borrowed item according to library rules.
- Hold request: place a request for a book currently issued to another borrower.
- Fine payment: record whether a fine associated with a loan has been paid.
- Collection management: allow a librarian to add, delete and update book records.

Administration: add clerks and librarians and view issued-book history and all books.

Limitations: The repository describes a console-based academic project. It does not document a web or mobile interface, online payment, barcode scanning, email notifications or cloud hosting as current features. These may be considered future enhancements.

1.4 SOFTWARE REQUIREMENTS SPECIFICATION

Architecture: client-server web application. The React frontend sends HTTP requests to the Express.js REST API, which validates the student's token, applies the business logic and reads or writes the database. Real-time chat messages are exchanged over a WebSocket connection, and the frontend updates the page as soon as new data arrives.

Hardware requirements

- Processor: dual-core or higher (suggested)
- RAM: 8 GB recommended to run Node.js, the database server, a browser and an IDE together
- Display: standard laptop or desktop

Software requirements

- Operating System: Windows, Linux or macOS compatible with the selected Java version.
- Programming Language: Java SE 8 (as specified by the original project instructions).
- IDE: NetBeans IDE.
- Database: Java DB / Apache Derby.
- Database Connectivity: JDBC.
- Diagram Tool: StarUML for the supplied .mdj class diagram.

Functional Requirements

- Search books by title, author and subject.
- Display borrower details and currently borrowed books.
- Allow authorized clerks to issue, return and renew books.
- Add borrowers and update borrower information.
- Record fine-payment status.
- Allow librarians to add, delete and update book records.
- Allow administrators to add staff and view library records.

Non-functional Requirements

- Usability: menu options and prompts should be clear.
- Reliability: issue and return transactions should be stored consistently.
- Maintainability: entities and operations should be separated into appropriate classes.
- Performance: common searches and updates should complete in reasonable time.

2. SYSTEM FLOW DIAGRAMS

2.1 USE CASE DIAGRAM

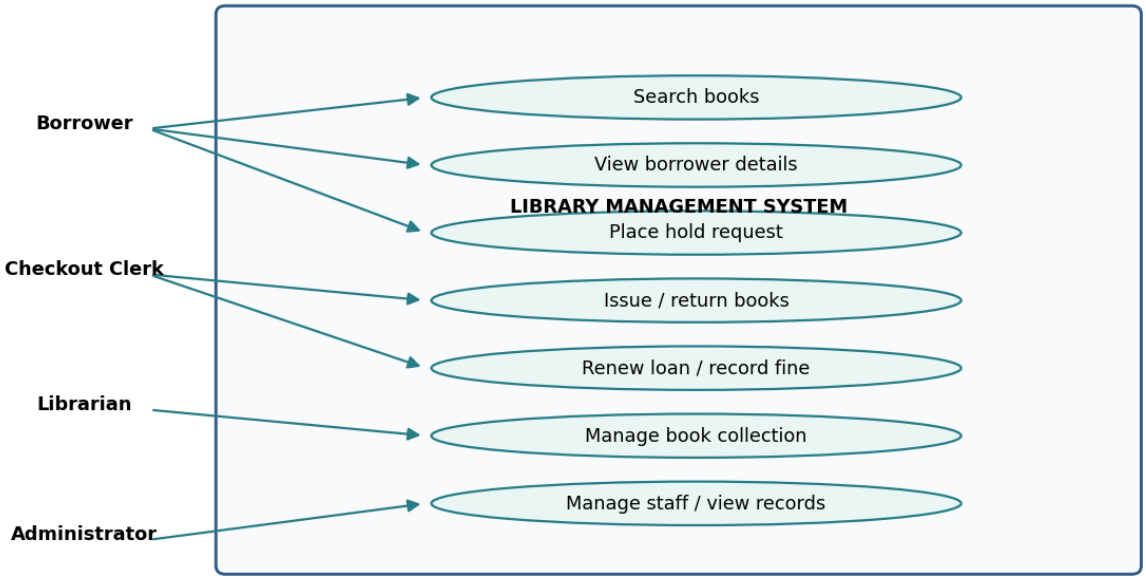


Figure 2.1: Student use cases

2.2 ENTITY RELATIONSHIP DIAGRAM

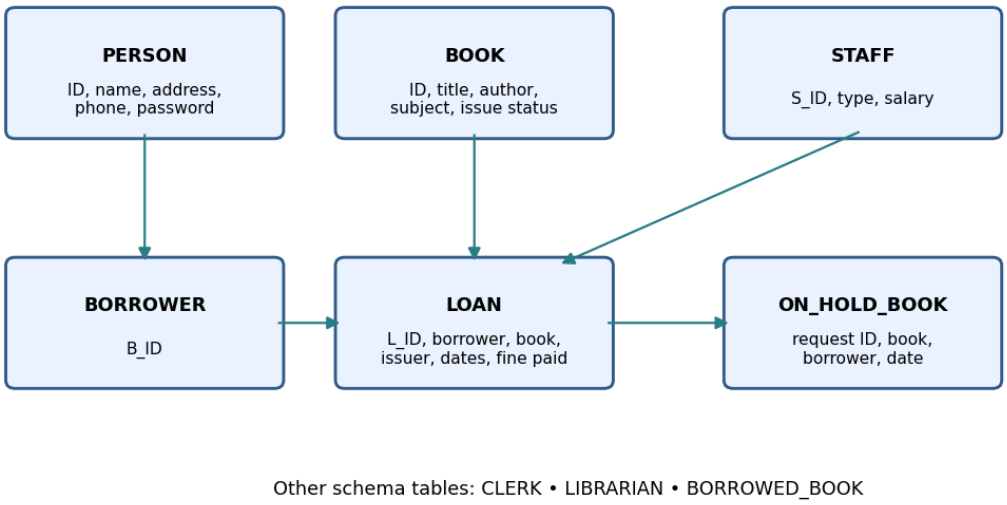


Figure 2.2: Main ShareNet entities stored in the MongoDB database

2.3 DATA FLOW DIAGRAM

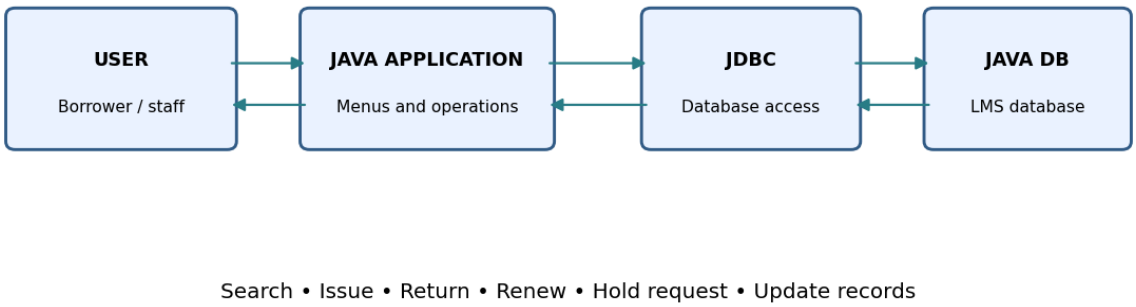


Figure 2.3: Browser, React frontend, Express API and database data flow

3. MODULE DESCRIPTION

Borrower Module

Supports searching books by title, author or subject, viewing personal information and checking currently borrowed books. A borrower can place a hold request when a required book is already issued.

Book Search Module

Searches BOOK records using title, author or subject. The issue-status field helps determine whether a book is currently issued.

Checkout and Return Module

Allows checkout clerks to issue books and receive returned books. A loan record stores borrower, book, issuing staff member and issue date. Return details include receiving staff and return date.

Loan Renewal Module

Allows a checkout clerk to renew an existing borrowed item according to library rules.

Borrower Management Module

Allows clerks to add a borrower and update personal information such as address and telephone number.

Fine Payment Module

Records whether a fine associated with a loan has been paid. The LOAN table contains a FINE_PAID field.

Book Collection Management Module

Allows librarians to add, delete and update book information. BOOK stores ID, title, author, subject and issue status.

Administration Module

Allows administrators to add checkout clerks and librarians and view issued-book history and all books.

Hold Request Module

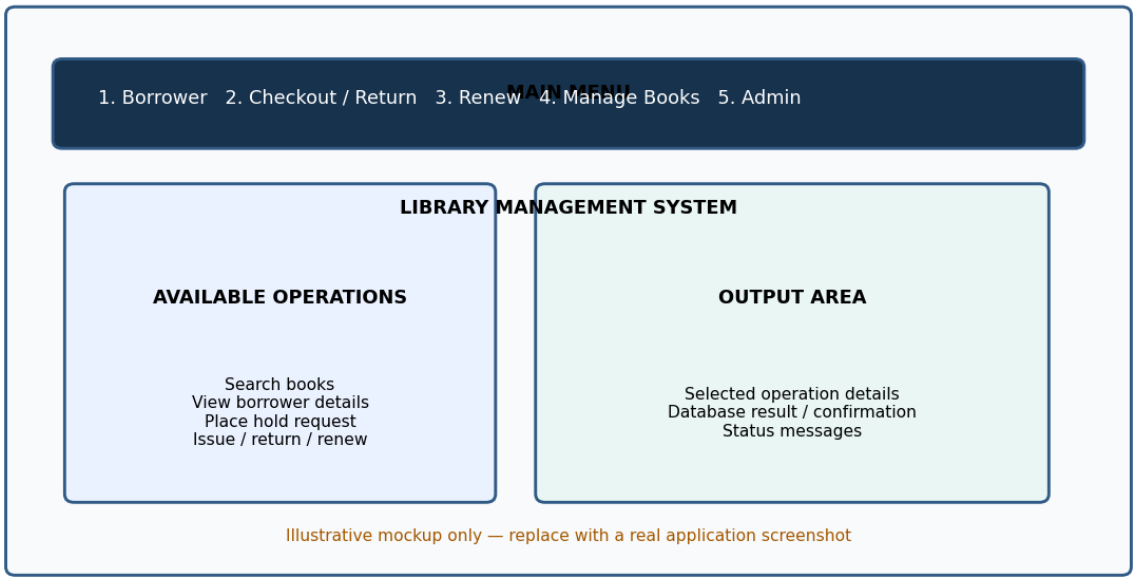
Maintains requests for books currently on loan. ON_HOLD_BOOK stores request ID, book, borrower and request date. The repository notes that HoldRequestOperations was introduced during refactoring to reduce bidirectional dependency between HoldRequest and Book.

4. IMPLEMENTATION

4.1 USER INTERFACE AND WORKFLOW

The original repository describes the interface as console-based. Users select the required operation from the menu. Available operations depend on the actor's role. The following workflow summarizes the documented setup and use.

- Install the required Java Development Kit and NetBeans IDE.
- In NetBeans, choose File → Open Project and select the downloaded Project folder.
- Configure Java DB by setting its database location to the Database folder supplied with the repository.
- Create a Java DB Network connection. The repository README specifies host localhost, port 1527 and database name LMS.
- Connect to the database before running the project.
- Select the required role or administrative function and use the menu options to search books, manage borrowers, issue or return books, renew loans or place hold requests.
- Verify the displayed result and corresponding database record after each operation.



4.2 DATA DESIGN

The application uses a relational database named LMS in Java DB (Derby). The supplied Database Schema.txt file defines the following tables:

Table	Purpose and important fields
PERSON	ID, PNAME, PASSWORD, ADDRESS and PHONE_NO.
BOOK	ID, TITLE, AUTHOR, SUBJECT and IS_ISSUED.
STAFF	S_ID, TYPE and SALARY.
CLERK	C_ID and DESK_NO.
LIBRARIAN	L_ID and OFFICE_NO.
BORROWER	B_ID.
BORROWED_BOOK	BOOK and BORROWER.
LOAN	L_ID, BORROWER, BOOK, ISSUER, ISS_DATE, RECEIVER, RET_DATE and FINE_PAID.
ON_HOLD_BOOK	REQ_ID, BOOK, BORROWER and REQ_DATE.

Primary keys uniquely identify records. Foreign keys maintain relationships between related tables. STAFF.S_ID and BORROWER.B_ID reference PERSON.ID. LOAN.BORROWER references BORROWER.B_ID, while LOAN.ISSUER and LOAN.RECEIVER reference STAFF.S_ID. CLERK.C_ID and LIBRARIAN.L_ID reference STAFF.S_ID.

4.3 SELECTED SOURCE CODE

The following short examples illustrate common Java/JDBC operations relevant to the system. They are report examples, not verified verbatim extracts from every source file in the repository. Compare them with your downloaded Project/src files and replace class or method names where necessary.

1. Establishing a JDBC connection

```
Connection con = DriverManager.getConnection(
    "jdbc:derby://localhost:1527/LMS",
    "haris", "123"
);
```

This example connects the Java application to the LMS database. The credentials are those documented in the public repository README; use your locally configured values.

2. Searching for a book

```
String sql = "SELECT * FROM BOOK WHERE TITLE = ?";
PreparedStatement ps = con.prepareStatement(sql);
ps.setString(1, bookTitle);
ResultSet rs = ps.executeQuery();
```

A prepared statement passes the book title as a parameter and returns matching records.

3. Updating book issue status

```
String sql = "UPDATE BOOK SET IS_ISSUED = ? WHERE ID = ?";
PreparedStatement ps = con.prepareStatement(sql);
ps.setBoolean(1, true);
ps.setInt(2, bookId);
ps.executeUpdate();
```

This example illustrates updating the issue status of a book. The actual application may perform this through its entity or operations classes.

4.4 TESTING SCENARIOS

Testing checks whether the main operations behave as expected. Execute these scenarios manually in the running application and verify the corresponding records in the LMS database.

Test ID	Action	Expected Result
T1	Search using a book title.	Matching book details are displayed.
T2	Search using an author name.	Books by the specified author are displayed.
T3	Search using a subject.	Books belonging to the subject are displayed.
T4	Issue an available book to a borrower.	A loan is recorded and issue status is updated.
T5	Attempt to issue a book already on loan.	The application prevents or reports the unavailable book.
T6	Return an issued book.	Return details are recorded and the book becomes available as per application logic.
T7	Renew a borrowed book.	The renewal is processed and updated details are retained.
T8	Place a hold request for an issued book.	A hold record is stored with book, borrower and request date.
T9	Add or update borrower information.	Borrower record is created or updated.
T10	Record fine payment.	Fine-payment status is updated.
T11	Add or update a book as librarian.	The collection reflects the changes.
T12	Add a clerk or librarian as administrator.	Staff record is created.

These are proposed manual test cases based on documented use cases and the database schema. Mark each test Pass or Fail only after running it. Add actual output screenshots for important test cases.

5. CONCLUSION

The Library Management System demonstrates the use of Java and object-oriented design concepts to support common library activities. It provides facilities for searching books, maintaining borrower information, issuing and returning books, renewing loans, placing hold requests and recording fine-payment details. The separation of user roles and application entities helps organize the system into manageable responsibilities.

JDBC and the relational Java DB database provide persistent storage for book, person, staff, borrower, loan and hold-request information. The project also provides a database schema and class diagram that help explain its structure. Overall, the system serves as an academic example of applying Java programming, database connectivity and object-oriented analysis and design to a practical problem.

Future enhancements may include a graphical user interface, barcode-based identification, automated fine calculation, email reminders, advanced search, report generation and migration to a modern database platform. These are suggested improvements and are not described as current features in the original repository.

6. REFERENCES

* Haris Muneer, “Library Management System – Java,” GitHub repository: * * *
https://github.com/harismuneer/Library-Management-System-JAVA
* “Database Schema.txt,” Library Management System – Java repository:
https://github.com/harismuneer/Library-Management-System-JAVA/blob/master/Database%20Schema.txt
* Oracle, Java Platform Documentation: https://docs.oracle.com/javase/
* Apache Derby Documentation: https://db.apache.org/derby/
* Oracle, JDBC Basics: https://docs.oracle.com/javase/tutorial/jdbc/
* Apache NetBeans Documentation: https://netbeans.apache.org/